6COSC022C.2 Advanced Server Side Web Programming Coursework Report

GitHub Link: https://github.com/Nipun23a/Advanced-Server-Side-Coursework-1.git

Name:J.A Nipuna Kaweya Prabashwara

Student ID:20221148/w1953831

Table of Contents

Introduction 3
System Overview 3
High-Level Architecture 4
MVC Architecture 6
Routing and API Design 8
Database Design 9
Load Balancing and Scalability 11
Security Architecture 12

Introduction

This project presents the design and implementation of a scalable Alumni Influencer Platform, developed to enhance engagement within an academic community. The system allows alumni to showcase their achievements and compete to become the featured influencer of the day through a structured blind bidding mechanism.

Traditional alumni platforms often lack interactive features and do not provide dynamic visibility for individual achievements. Additionally, there is limited support for structured promotion, sponsorship opportunities, and fair selection mechanisms. To address these challenges, this system introduces a competitive and transparent bidding model, where alumni can place bids using available sponsorship funds. The highest valid bid is selected daily, ensuring fairness while maintaining user engagement.

The platform is built using a hybrid architecture that separates responsibilities across multiple layers. The web application is developed using CodeIgniter 4, which follows the Model-View-Controller (MVC) pattern for handling user interfaces, authentication, and profile management. The business logic and API layer are implemented using Express.js, enabling scalable and modular handling of bidding, sponsorship, and public API services. A MySQL database is used for persistent storage, ensuring data consistency and relational integrity across all system components.

To support scalability and reliability, the system incorporates a load-balanced architecture, allowing multiple instances of both the web and API layers to handle incoming traffic efficiently. Additional features such as API key management, rate limiting, and secure authentication mechanisms are integrated to ensure system security and controlled access.

Overall, the platform demonstrates a modern, modular, and scalable approach to building web applications, combining traditional MVC principles with service-oriented architecture to deliver a flexible and high-performance solution.

System Overview

The Alumni Influencer Platform is designed as an interactive web-based system that enables alumni to actively participate in a competitive and engaging environment. The platform

integrates profile management, sponsorship mechanisms, and a blind bidding system to determine a daily features alumni influencer.

The system provides several key functionalities.First,it support secure user authentication and profile management, allowing alumni to maintain detailed professional profiles that include degrees, certification, licenses, short professional courses, and employment history. These profiles serve as the foundation for showcasing individual achievements.

A central feature of the system is the blind bidding mechanism**,** where users place bids without visibility into competing bids. This ensures fairness and prevents bias in the selection process. The system automatically selects the highest valid bid at the end of each day, designating the winning user as the featured alumni. To maintain fairness, constraints such as monthly participation limits and event-based eligibility are enforced.

The platform also incorporates a sponsorship model**,** where sponsors such as companies, universities, and certification providers can fund alumni activities. These sponsorship offers can be utilized by alumni to place bids, creating a structured and realistic funding ecosystem within the platform.

In addition, the system exposes a set of public API endpoints that allow external clients, such as mobile applications or augmented reality (AR) platforms, to access featured alumni data. This enables integration with external systems and enhances the visibility of alumni achievements beyond the core platform.

Overall, the system combines user interaction, automated decision-making, and external integration to deliver a dynamic and scalable solution that encourages continuous alumni engagement.

High-Level Architecture

The system is designed using a layered approach that separates the web interface, backend services, and data storage. This makes the application easier to manage, scale, and extend in the future.

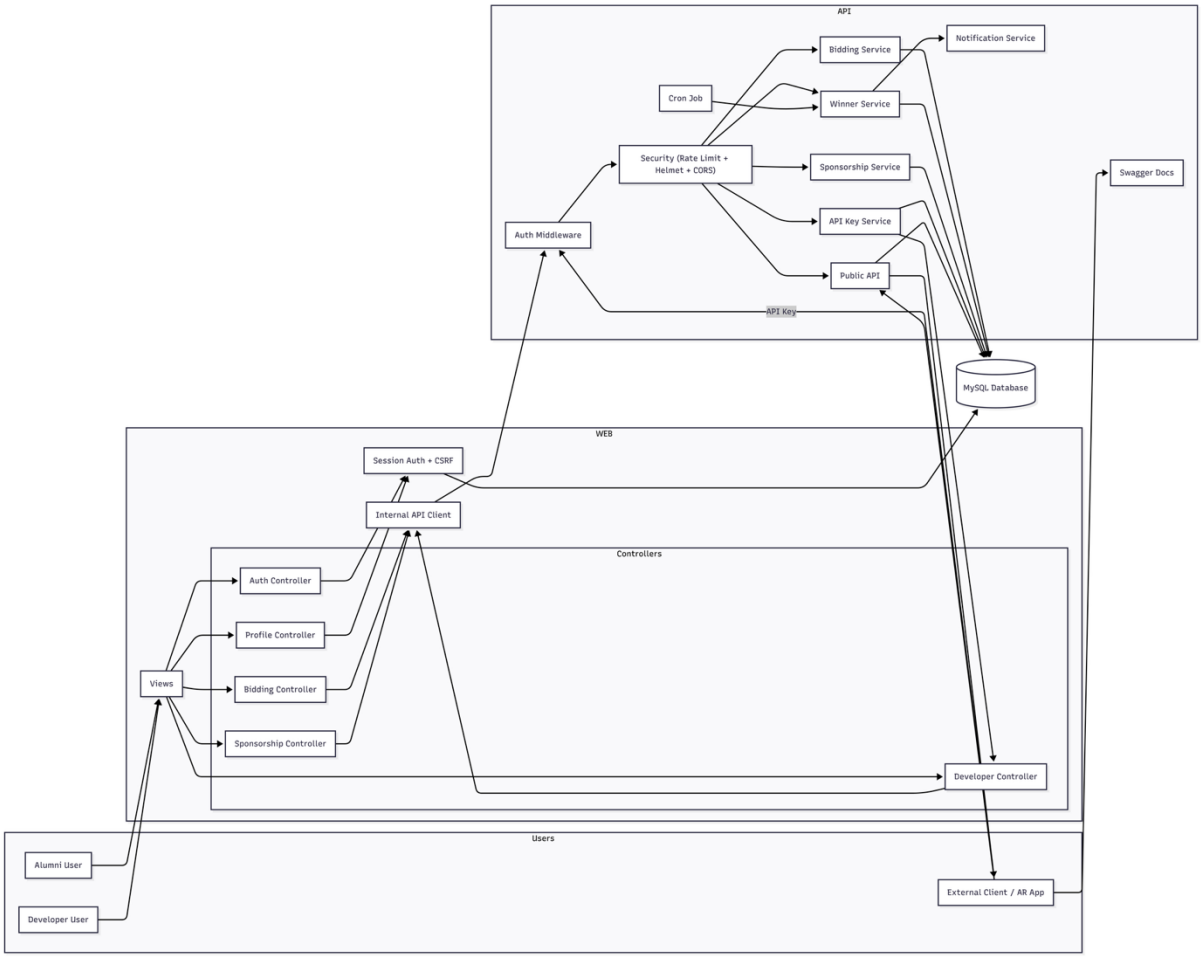

Figure 1:High-Level Architecture Diagram

At the top level, users interact with the system through the web application built using CodeIgniter 4. This layer handles user-facing features such as login, profile management, bidding, and sponsorship interactions. It follows the MVC pattern, where views present data to users and controllers handle incoming requests.

Instead of placing all logic inside the web application, the system uses an internal API client to communicate with a separate backend built using Express.js. This backend is responsible for handling the main business logic, including bidding operations, sponsorship processing, and selecting daily winners. By separating these responsibilities, the system remains clean and easier to maintain.

Before any request reaches the core services, it passes through security components such as authentication middleware and additional protections like rate limiting and CORS. This ensures that only valid and authorized requests are processed.

The API layer is organized into multiple services, each handling a specific function. For example, the bidding service manages user bids, the winner service determines the daily featured alumni, and the sponsorship service handles funding-related operations. A scheduled cron job runs automatically to select the daily winner, and a notification service is used to inform users about important updates.

Both the web and API layers rely on a shared MySQL database to store and retrieve data. This includes user accounts, profiles, bids, sponsorships, and system logs. Keeping a centralized database ensures consistency across the system.

In addition, the system provides public API endpoints that allow external applications, such as mobile or AR platforms, to access featured alumni information. These endpoints are secured using API keys to control access and usage.

Overall, this architecture balances simplicity and scalability by combining a traditional web framework with a separate service-based backend. This makes the system flexible enough to handle growth while keeping the codebase organized and maintainable.

MVC Architecture

The system uses the Model-View-Controller (MVC) pattern in the CodeIgniter 4 web layer to organize the application in a structured way. This helps to separate the user interface, application logic, and data handling, making the system easier to manage and update.

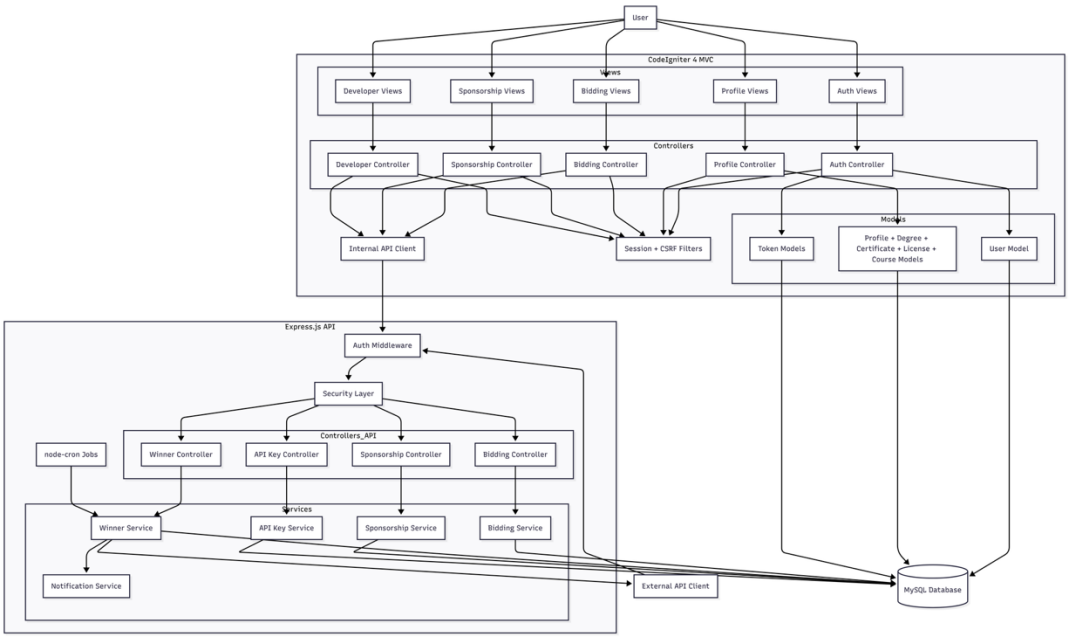

Figure 2:MVC Diagram

At the top level, users interact with the system through different views such as authentication pages, profile pages, bidding pages, sponsorship pages, and developer dashboards. These views are responsible for displaying data and collecting user input, but they do not contain any business logic.

When a user performs an action, the request is handled by the corresponding controller. For example, the Auth Controller manages login and registration, the Profile Controller handles user details, and the Bidding and Sponsorship Controllers manage their respective features. Controllers act as the middle layer that connects the views with the rest of the system.

Inside the web layer, controllers interact with models to retrieve or store data in the database. Models represent the data structure of the system, such as users, profiles, tokens, and other related entities. This keeps all database-related operations organized and separate from the user interface.

In addition to working with models, some controllers also communicate with the backend API through an internal API client. This is used when handling more complex operations such as bidding and sponsorship logic. Instead of placing all logic inside the web layer, these operations are delegated to the API layer, which makes the system more modular and easier to scale.

Security is also integrated within the MVC flow. Session-based authentication and CSRF protection are applied to ensure that only authorized users can access certain features and that requests are safe.

Overall, the MVC structure helps keep the application clean and organized. By separating responsibilities between views, controllers, and models, the system becomes easier to maintain, test, and extend in the future.

Routing and API Design

The system uses a structured routing approach to handle requests from both users and external applications. The routing flow is divided into two main parts: the CodeIgniter web routes and the Express.js API routes.

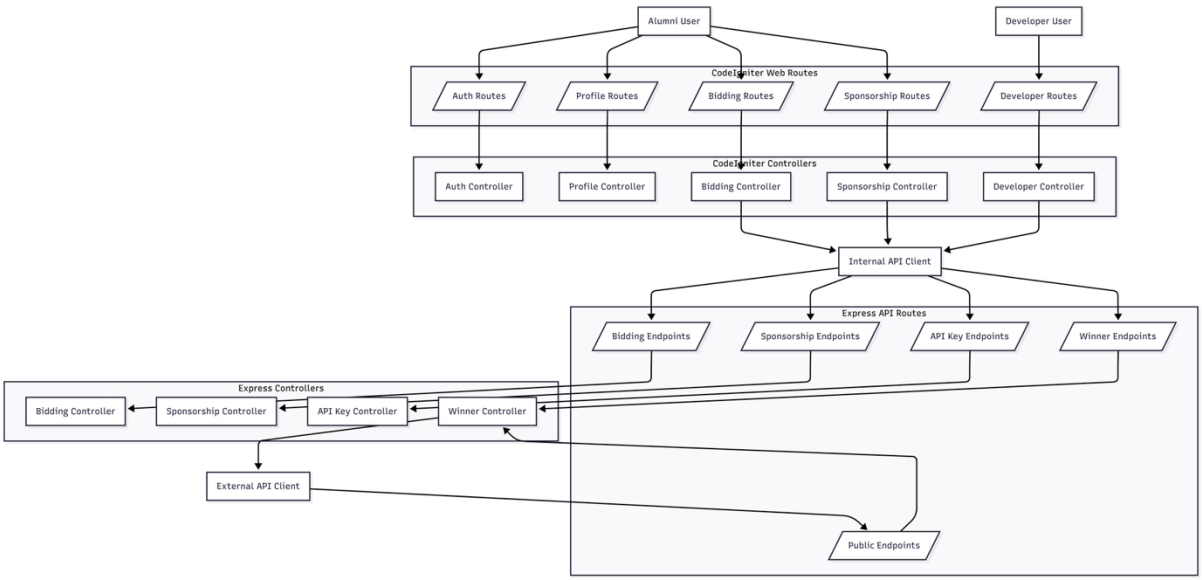

Figure 3:Routing and API Design

On the web side, users interact with different routes such as authentication, profile management, bidding, sponsorship, and developer features. Each of these routes is connected to a specific controller in the CodeIgniter application. For example, authentication routes are handled by the Auth Controller, while bidding-related requests go to the Bidding Controller. This keeps the routing organized and easy to manage.

When simple operations are needed, controllers directly handle the request using models. However, for more complex features like bidding, sponsorship, and API-related actions, the

controllers use an internal API client to send requests to the Express backend. This helps keep the web layer lightweight and focused only on handling user interaction.

On the backend side, the Express.js API is organized into multiple endpoint groups. These include bidding endpoints, sponsorship endpoints, API key endpoints, and winner endpoints. Each group is connected to its own controller, which processes the request and passes it to the relevant service.

The system also supports public endpoints that can be accessed by external clients such as mobile apps or AR applications. These endpoints allow external systems to retrieve data like the featured alumni. Access to these endpoints is controlled using API keys to ensure security.

Overall, the routing design clearly separates responsibilities between the web layer and the API layer. This makes the system easier to maintain, improves scalability, and allows new features to be added without affecting existing functionality.

Database Design

The system uses a relational database designed to organize data in a structured and efficient way. The database is built using MySQL and follows a normalized approach to reduce redundancy and maintain data consistency.

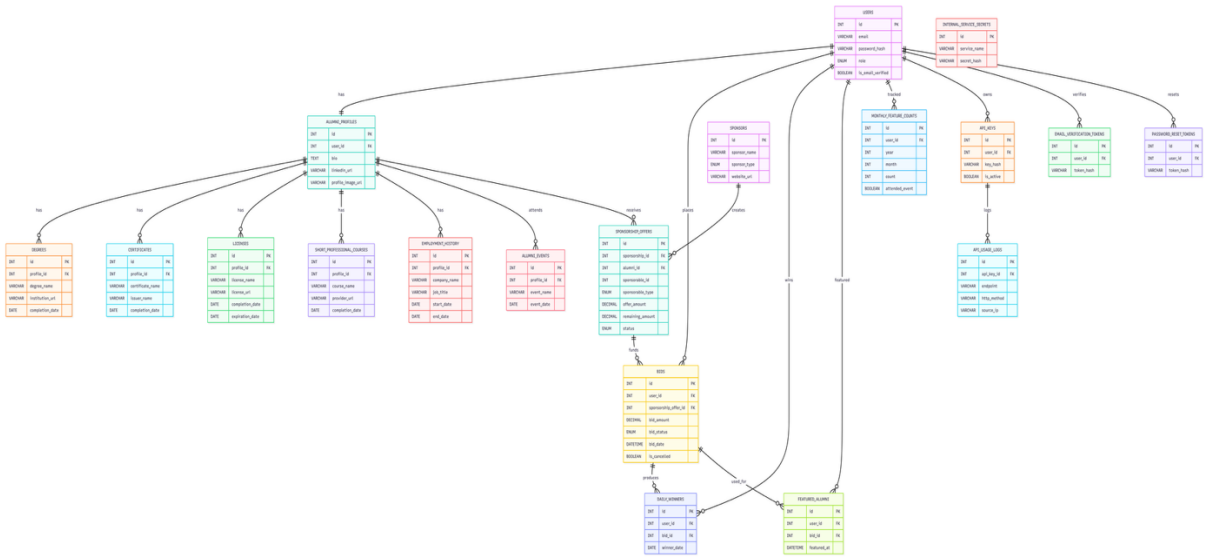

Figure 4:Database ER-Diagram

At the core of the system is the users table, which stores basic information such as email, password, and user role. Each user is linked to an alumni profile, which contains additional details like bio, LinkedIn URL, and profile image. This separation helps keep user authentication data and profile information organized.

The alumni profile is connected to several other tables that store professional and academic details. These include degrees, certificates, licenses, short professional courses, and employment history. Each of these tables is linked to the profile using foreign keys, allowing a user to have multiple records in each category.

The system also includes a bidding module, which is a key part of the platform. The bids table stores all bid-related information such as bid amount, status, and date. Each bid is linked to a user and can also be connected to a sponsorship offer. The daily_winners table records the winning bid for each day, while the featured_alumni table stores information about users who have been selected as featured.

To manage fairness, the system uses a monthly_feature_counts table to track how many times a user has won within a month. This helps enforce rules such as limiting the number of wins and allowing additional participation only under certain conditions, like attending events. The alumni_events table is used to store event participation data.

The sponsorship system is handled through the sponsors and sponsorship_offers tables. Sponsors can create offers that alumni can use when placing bids. This adds a realistic funding mechanism to the platform and connects sponsorships directly with user activity.

For security and system management, additional tables are included such as API keys, API usage logs, email verification tokens, password reset tokens, and internal service secrets. These tables help manage authentication, track usage, and maintain system security.

Overall, the database is designed to support all core features of the system while keeping relationships clear and organized. By using foreign keys and separating data into multiple related tables, the system ensures consistency and makes it easier to manage and scale in the future.

Load Balancing and Scalability

To improve performance and handle multiple users efficiently, a load balancing approach is proposed for this system. Instead of running the application on a single server, the system is designed to distribute incoming traffic across multiple instances. This helps avoid bottlenecks and ensures the system remains stable even when the number of users increases.

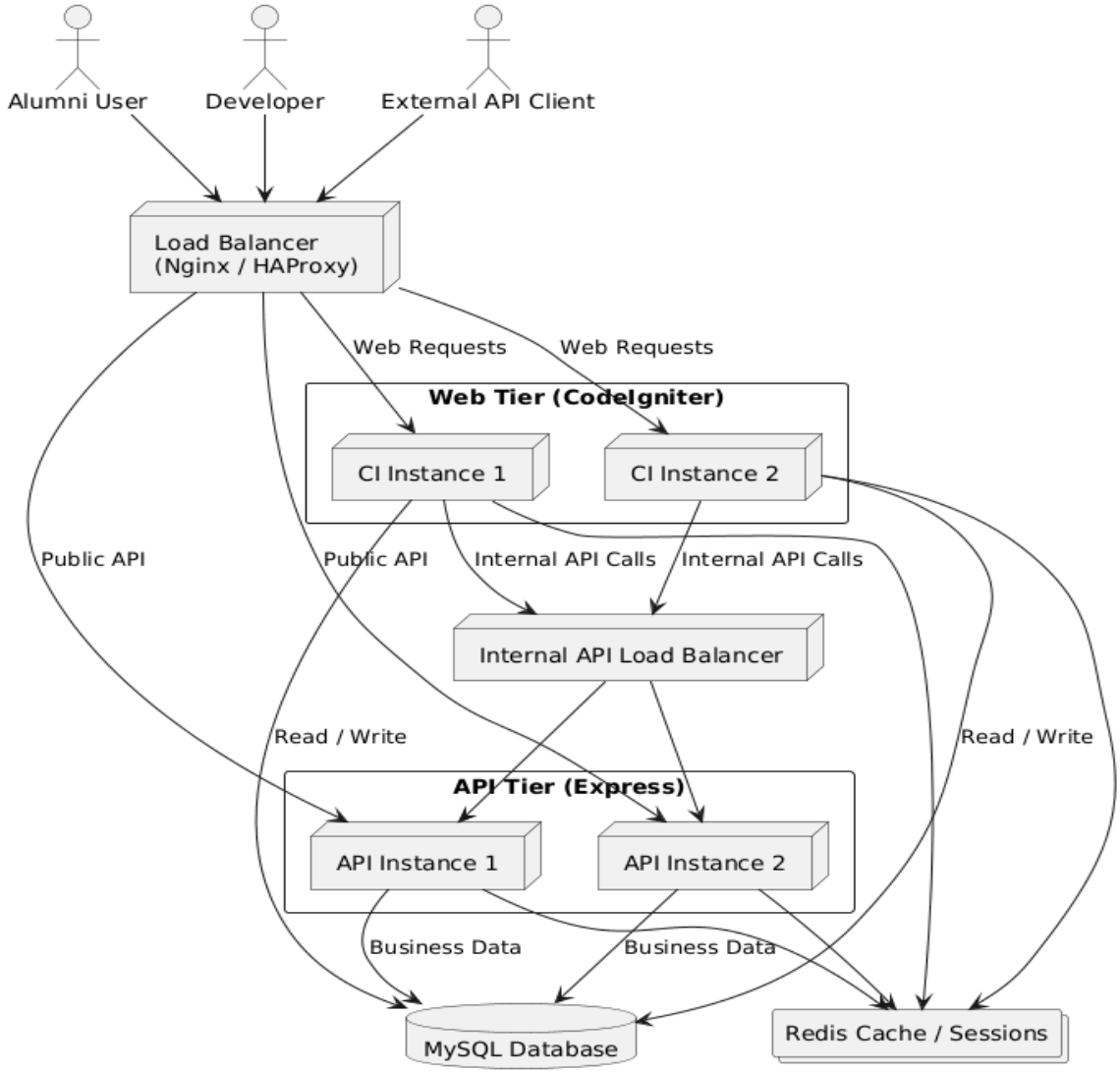

Figure 5:Load Balancing and Scalability Diagram

As shown in the diagram above, all incoming requests from alumni users, developers, and external API clients first go through a load balancer. This load balancer (such as Nginx or HAProxy) distributes the traffic between multiple CodeIgniter web instances. By doing this, the system can handle more users at the same time without slowing down.

The web layer then communicates with the backend API through an internal API load balancer. This ensures that API requests are also distributed across multiple Express.js instances. As a result, operations like bidding, sponsorship handling, and winner selection can run smoothly without overloading a single server.

Both the web layer and API layer share the same MySQL database, which is used to store all system data. In addition, a Redis cache layer is included to improve performance by handling sessions and frequently accessed data more efficiently.

Overall, this load balancing setup is a proposed design that prepares the system for future growth. It allows the application to scale horizontally, improve reliability, and provide a better experience for users even under high traffic conditions.

Security Architecture

The system is designed with multiple layers of security to protect user data and prevent common web attacks. Instead of relying on a single method, different security mechanisms are applied across both the web application and the API. This layered approach helps reduce risks and improves overall system safety.

At the authentication level, the system uses different methods depending on the type of user. Alumni users who access the system through the web use session-based authentication, while external clients use API keys in the form of bearer tokens. Communication between the CodeIgniter web layer and the Express API is secured using an internal secret. This ensures that only trusted internal requests are allowed to access the backend services.

Passwords are handled securely using hashing. When a user registers, their password is validated for strength and then hashed using bcrypt before being stored in the database. During login, the entered password is compared with the stored hash. This means that raw passwords are never stored, reducing the risk of data leaks.

The system also uses secure token handling for features such as email verification, password reset, API keys, and internal secrets. Tokens are generated using secure random methods and stored in the database as hashed values instead of plain text. They are also time-limited and can only be used once, which adds an extra layer of protection.

Input validation is applied throughout the system to prevent invalid or harmful data from being processed. Both the web layer and API validate inputs such as numbers, dates, and text fields. For example, bid amounts must be positive values, and all IDs must be valid integers. This helps protect the system from attacks such as SQL injection and malformed requests.

Additional security measures include the use of HTTP security headers, which are applied to prevent issues like cross-site scripting (XSS) and clickjacking. CSRF protection is enabled in the web layer to secure all form submissions. On the API side, rate limiting is used to control how many requests a user can make within a certain time period, helping to prevent abuse and brute-force attacks.

Access control is also enforced to ensure that users can only perform actions they are allowed to. For example, some API routes are restricted to internal use only, while others are accessible to external clients with valid API keys. Users must also verify their email before accessing certain features.

Finally, the system includes logging and monitoring features. API usage is tracked, and logs are generated for incoming requests and system activity. This helps detect unusual behavior and provides useful information for debugging and security analysis.

Overall, the security design follows a layered approach, combining authentication, validation, encryption, and monitoring to protect the system from different types of threats.